

UNIVERSITY OF CENTRAL PUNJAB

Faculty of Information Technology

Student Internship & Job Tracking System

A Containerized Full-Stack Web Application for Placement Lifecycle &
Analytics Pipeline Audit

Course:Tools & Techniques for Data Science

Program:Bachelor of Science in Data Science (6th Semester)

Project Type:Reproducible Data Applications (Assignment #4)

Submitted By:Muhammad Zain

Group Members:Muhammad Zain 0060, Muhammad Nabeel Arshad 0074, Razi
U Din 0063

Project Supervisor:Prof. Muhammad Rizwan

Submission Date:June 2026

1. Executive Summary

In modern educational management, coordinating corporate alignment, tracking student recruitment phases, and auditing job market shifts present major structural difficulties. This report presents the **Student Internship & Job Tracking System**, a comprehensive data application engineered explicitly as a secure Faculty Portal. Built using Python, Streamlit, and PostgreSQL, this system introduces a resilient relational architecture containerized completely within Docker networks to achieve total reproducibility across environments.

The portal empowers departmental heads and placement coordinators to record incoming recruitment leads, execute filtered granular multi-keyword searches, track application lifecycles dynamically, and derive immediate industrial hiring insights through automated analytics dashboards. Crucially, the solution solves the problem of cross-machine dependency conflicts by locking runtime environments into microservices, ensuring that database updates, schema creation, and persistent volume mount structures operate identically across any hosting infrastructure.

2. System Architecture & Containerization Layout

To fulfill academic reproducibility standards, the application relies on an isolated multi-container architecture orchestrated via Docker Compose. Isolation guarantees structural resilience, while network bridge exposure ensures highly structured, secure interaction between separate runtime components.

Service Name	Base Image / Technology	Port Mapping	Core Functionality Description
opportunity_streamlit	Python 3.10-slim / Streamlit Framework	8501:8501	Manages user interaction, reactive session processing, dynamic form validation, and multi-page view

Service Name	Base Image / Technology	Port Mapping	Core Functionality Description
			rendering.
opportunity_postgres	PostgreSQL 15-alpine Relational Engine	5432:5432	Handles persistent relational records, executes multi-filter searches, enforces structural constraints, and provides persistent storage via mounted volumes.
opportunity_pgadmin	dpage/pgadmin4 (Web Console)	5050:80	Enables graphic data auditing, database inspection, ad-hoc execution tracking, and schema health validation.

Data integrity is maintained through a secure external Docker Volume called `postgres_data` mapped to the path `/var/lib/postgresql/data`. This configuration ensures that if containers are stopped, modified, or rebuilt using `docker compose down`, placement records remain intact.

3. Database Design & Relational Schema

The backend engine depends on a strictly normalized relational schema constructed in PostgreSQL. The target table, titled `opportunities`, leverages an auto-incrementing surrogate primary key to securely reference specific rows. This structural design prevents data parsing conflicts encountered in non-relational configurations.

3.1 SQL Schema Specification (DDL)

The operational structural schema executed inside the initialization layer is defined as follows:

```
CREATE TABLE opportunities (  
    opportunity_id SERIAL PRIMARY KEY,  
    company_name VARCHAR(255) NOT NULL,  
    job_title VARCHAR(255) NOT NULL,  
    category VARCHAR(100) NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    country VARCHAR(100) NOT NULL,  
    work_mode VARCHAR(50) DEFAULT 'Onsite',  
    required_skills TEXT,  
    salary_min NUMERIC(12, 2) DEFAULT 0.0,  
    salary_max NUMERIC(12, 2) DEFAULT 0.0,  
    currency VARCHAR(10) DEFAULT 'PKR',  
    application_deadline DATE,  
    experience_level VARCHAR(50) DEFAULT 'Entry Level',  
    status VARCHAR(50) DEFAULT 'Open',  
    source_link TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4. Application Modules Implementation (CRUD & Advanced Analytics)

The software deployment features a structured multi-page layout accessible via a secure sidebar panel, breaking down operations into distinct, isolated modules:

- **Add Opportunity (Create):** Provides an interactive validation form where administrators enter hiring metrics. The form enforces data validation rules before initiating secure parameterized database insertions.
- **View & Search Engine (Read):** Integrates case-insensitive pattern matching via PostgreSQL ILIKE operators. Faculty can filter data by skill keywords, minimum salary thresholds, target locations, or employment status.
- **Update Opportunity (Update):** Resolves state tracking mismatches by binding modifications to the unique primary key opportunity_id, allowing real-time status shifts and

structural work-mode alterations.

- **Delete Opportunity (Delete):** Permits hard deletion of stale or invalid data using isolated transactional statements to safeguard operational integrity.
- **Analytics Dashboard:** Pulls dynamic aggregation queries (such as AVG, COUNT, and ROUND) to display real-time Key Performance Indicators, highlighting active placements, average localized salary distributions, and highly demanded skill-sets.
- **Duplicate Detection System:** Identifies matching patterns by grouping clusters by company name, job title, and source link, using ARRAY_AGG to surface redundant records.

5. Technical Verification & Integrity Diagnostics

During deployment phases, initial issues involving positional argument mismatches in status modification updates and missing library references were successfully resolved by rewriting the query engine functions to leverage flexible Python keyword argument wrappers (*args and **kwargs).

Database operations were tested against unexpected column names, verifying that explicit references to opportunity_id instead of generic id parameters prevent transactional failures. System tests confirm successful multi-container synchronization, verified via clean container rebuild diagnostics:

```
$ docker compose down
$ docker compose up --build -d
✓ Network streamlit-postgres-assignment_default    Created
✓ Container opportunity_postgres                  Started
✓ Container opportunity_pgadmin                    Started
✓ Container opportunity_streamlit                  Started
pgAdmin SQL Verification Logs
```

6. pgAdmin SQL Verification Logs :

Query 1:

localhost:5050/browser/

Admin | File | Object | Tools | Help | admin@example.com (internal)

Object Explorer | Query | Properties | SQL | Statistics | Dependencies | Dependents | Processes | student_opportunities_db/app_user@Muhammad Zain

Query: `SELECT * FROM opportunities;`

Data Output: Showing rows: 1 to 45 | Page No: 1 of 1

opportunity_id [PK] integer	company_name character varying (100)	job_title character varying (150)	category character varying (50)	city character varying (80)	country character varying (80)	work_mode character varying (30)
1	Systems Limited	Junior Data Scientist	Data Science	Lahore	Pakistan	Onsite
2	Systems Limited	Senior ML Engineer	AI	Lahore	Pakistan	Hybrid
3	Systems Limited	Full Stack Developer	Web Development	Lahore	Pakistan	Onsite
4	Systems Limited	Cybersecurity Analyst	Cyber Security	Lahore	Pakistan	Onsite

Total rows: 45 | Query complete 00:00:00.348 | CRLF | Ln 1, Col 29

Query 2 :

Admin | File | Object | Tools | Help | admin@example.com (internal)

Object Explorer | Query | Properties | SQL | Statistics | Dependencies | Dependents | Processes | student_opportunities_db/app_user@Muhammad Zain

Query: `SELECT COUNT(*) FROM opportunities;`

Data Output: Showing rows: 1 to 1 | Page No: 1 of 1

count bigint
45

Query 3 :

pgAdmin

File Object Tools Help

admin@example.com (internal)

student_opportunities_db/app_user@Muhammad Zain

Query

```
1 SELECT category, COUNT(*)
2 FROM opportunities
3 GROUP BY category;
```

Data Output

category	count
Data Science	15
Cyber Security	6
Web Development	7
Software Engineering	8
AI	9

Showing rows: 1 to 5

Page No: 1 of 1

Total rows: 5 Query complete 00:00:00.159

CRLF Ln 3, Col 19

Query 4:

pgAdmin

File Object Tools Help

admin@example.com (internal)

student_opportunities_db/app_user@Muhammad Zain

Query

```
1 SELECT work_mode, COUNT(*)
2 FROM opportunities
3 GROUP BY work_mode;
```

Data Output

work_mode	count
Remote	11
Hybrid	16
Onsite	18

Showing rows: 1 to 3

Page No: 1 of 1

Total rows: 3 Query complete 00:00:00.257

CRLF Ln 3, Col 20

Query 5:

The screenshot shows the pgAdmin interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT * FROM opportunities
2 WHERE status = 'Open';
```

The results are displayed in the 'Data Output' tab, showing 35 rows. The columns are: opportunity_id (PK integer), company_name (character varying (100)), job_title (character varying (150)), category (character varying (50)), city (character varying (80)), country (character varying (80)), and work_mode (character varying (30)).

opportunity_id	company_name	job_title	category	city	country	work_mode
1	Systems Limited	Junior Data Scientist	Data Science	Lahore	Pakistan	Onsite
2	Systems Limited	Senior ML Engineer	AI	Lahore	Pakistan	Hybrid
3	Systems Limited	Cybersecurity Analyst	Cyber Security	Lahore	Pakistan	Onsite
4	Systems Limited	Data Engineer Intern	Data Science	Lahore	Pakistan	Hybrid
5	Systems Limited	Software Engineer / SDET	Software Engineering	Lahore	Pakistan	Onsite

Query 6 :

The screenshot shows the pgAdmin interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT * FROM opportunities
2 WHERE application_deadline <= CURRENT_DATE + INTERVAL '7 days';
```

The results are displayed in the 'Data Output' tab, showing 19 rows. The columns are: opportunity_id (PK integer), company_name (character varying (100)), job_title (character varying (150)), category (character varying (50)), city (character varying (80)), country (character varying (80)), work_mode (character varying (30)), and request_type (text).

opportunity_id	company_name	job_title	category	city	country	work_mode	request_type
1	Systems Limited	Junior Data Scientist	Data Science	Lahore	Pakistan	Onsite	Pyth
2	Systems Limited	Full Stack Developer	Web Development	Lahore	Pakistan	Onsite	React
3	Systems Limited	Data Engineer Intern	Data Science	Lahore	Pakistan	Hybrid	Pyth
4	Systems Limited	AI Research Intern	AI	Lahore	Pakistan	Remote	Pyth
5	Systems Limited	Data Analyst	Data Science	Lahore	Pakistan	Hybrid	Pyth

6. Conclusion & Recommendations

The Student Internship & Job Tracking System successfully demonstrates how containerized application architectures can simplify departmental data tracking. Moving database interactions

from fragmented spreadsheet environments into a relational schema wrapped in an automated UI establishes an audited pipeline.

For future iterations, we recommend adding automated background data scrapers (e.g., using n8n workflows or scheduled cron tasks) to dynamically ingest real-time market openings directly into the PostgreSQL engine, alongside automated user activity logging for enhanced audit control.